

Emotion Detection in Text Using Fine-Tuned BERT on GoEmotions

Deep Learning for NLP — Course Project

Assil Halawi

Lebanese American University

Student ID: 202300403

May 1, 2026

Abstract

This project focuses on emotion detection in text using a fine-tuned BERT model. The goal was to classify each sentence into one of six emotions: sadness, love, joy, surprise, anger, and fear. The original GoEmotions dataset is multi-label, meaning that one sentence can express more than one emotion at the same time, so the task was simplified into a single-label classification problem by selecting the first active emotion for each instance. After preprocessing the dataset, filtering it to the chosen six emotions, tokenizing the text with the BERT tokenizer, and fine-tuning a pretrained BERT model, the final system achieved an accuracy of 0.7708 and a weighted F1-score of 0.7698 on the test set. A DistilBERT model was also trained under identical conditions as a baseline, achieving 0.7682 accuracy and 0.7669 weighted F1, confirming that full BERT provides a slight but consistent advantage. The results show that BERT performs well overall for emotion classification, while still struggling on classes that are semantically similar or underrepresented.

Contents

1	Introduction	2
2	Project Objective	2
3	Dataset Description	2
4	From Multi-Label to Single-Label Classification	3
5	Implementation Environment	3
6	Data Preprocessing	4
7	Train, Validation, and Test Split	5
8	Why BERT Was Chosen	5
9	Tokenization	5
10	Creating the Dataset Objects	6
11	Fine-Tuning the Pretrained BERT Model	6
12	Training Process	7
13	Evaluation Metrics	7
14	Final Results	8
15	Interpretation of the Results	9
16	Confusion Matrix Analysis	9
17	Error Analysis	10
18	Baseline Comparison: BERT vs. DistilBERT	11
19	What Was Successfully Achieved	13
20	Limitations	14
21	Possible Improvements	14
22	Conclusion	15
	Appendix: Summary of the Full Implementation Flow	16

1. Introduction

Emotion detection is an important Natural Language Processing task that aims to identify the emotional content expressed in text. Unlike simple sentiment analysis, which usually predicts only positive, negative, or neutral sentiment, emotion detection is more fine-grained because it attempts to distinguish between emotions such as joy, sadness, anger, fear, surprise, and love. This makes the task more useful in real-world applications such as social media monitoring, conversational agents, mental health analysis, customer feedback understanding, and human-computer interaction.

The purpose of this project was to build an emotion classification system using deep learning for NLP, specifically a transformer-based model. The chosen model was BERT, which is one of the most important pretrained transformer models used in modern NLP. The project used the GoEmotions dataset and adapted it to a simplified single-label setting in order to make the classification process easier to train, evaluate, and interpret.

2. Project Objective

The main objective of the project was to classify text into six emotional categories: sadness, love, joy, surprise, anger, and fear.

The project also aimed to:

1. Use the GoEmotions dataset as the source of labeled text.
2. Convert the dataset from multi-label format to single-label format.
3. Fine-tune a pretrained BERT model on the processed dataset.
4. Evaluate the model using accuracy and F1-score.
5. Analyze the model's strengths and weaknesses using per-class metrics and a confusion matrix.
6. Perform error analysis to understand which emotions are harder to classify and why.
7. Compare the fine-tuned BERT model against a DistilBERT baseline.

3. Dataset Description

The dataset used in this project was the GoEmotions dataset. GoEmotions is a well-known dataset for emotion analysis that contains text samples labeled with emotions. In its original form, the dataset is multi-label, meaning a single sentence may have multiple emotions assigned to it at the same time.

For example, one sentence might express both sadness and anger, or joy and surprise. In the dataset file used for this project, the data was stored in a one-hot encoded tabular

format. Each row contained a text sample and multiple emotion columns, with each column containing either 0 or 1. A value of 1 in an emotion column meant that the corresponding emotion was present in that sentence.

The dataset contained many emotion columns, but this project focused only on six classes: sadness, love, joy, surprise, anger, and fear. This selection reduced the difficulty of the problem and made the final model easier to interpret.

4. From Multi-Label to Single-Label Classification

One of the most important design choices in this project was converting the original multi-label dataset into a single-label dataset.

In the original GoEmotions format, a sentence could contain more than one active emotion. However, for this project, each sentence had to belong to one and only one class so that the task could be handled as a standard classification problem using BERT.

The conversion was performed by selecting the first active emotion in the dataset's emotion columns. This was done programmatically using Python with the `idxmax(axis=1)` operation over the emotion columns. Since the emotion columns contained 0s and 1s, this operation returned the name of the first column with a value of 1 for each row. That selected emotion became the final label for that sentence.

It is important to note that this simplification makes the task easier, but it also introduces a limitation. Some sentences naturally contain multiple emotions, and forcing them into a single class removes part of the emotional information. In addition, selecting the first active label may introduce bias based on the order of the columns rather than the true strength of the emotions.

5. Implementation Environment

The project was implemented in Google Colab. Colab was a suitable choice because it supports Python notebooks, deep learning libraries, and GPU acceleration. This made it possible to train a transformer-based model efficiently without requiring local hardware setup.

The main libraries used were:

- **pandas** — for loading and manipulating the dataset.
- **scikit-learn** — for splitting the dataset and computing evaluation metrics.
- **transformers** (Hugging Face) — for the BERT tokenizer, BERT model, training utilities, and fine-tuning.
- **torch** — for dataset handling and model training.

- **matplotlib** and **seaborn** — for visualization of the confusion matrix.

6. Data Preprocessing

The implementation began by uploading the CSV file into Google Colab and loading it with pandas.

After loading the dataset, the first rows were displayed to confirm that the file structure was correct. The text column was present, and the remaining columns represented emotion labels in one-hot encoded form.

The emotion columns were identified by excluding the initial non-emotion metadata columns and selecting all remaining columns from the fourth column onward. Then, the single-label conversion step was applied using `idxmax(axis=1)` to create a new label column containing one final emotion name per row.

Once the single-label conversion was complete, the dataset was filtered to retain only rows whose resulting label belonged to the six selected emotions: sadness, love, joy, surprise, anger, and fear. After this filtering step, only the text and label columns were kept, producing a clean dataset specifically tailored to the final classification task.

Next, the textual labels were converted into numeric class IDs, since BERT classification models require integer labels. The label mapping produced during the run was:

Emotion	Class ID
sadness	0
love	1
joy	2
surprise	3
anger	4
fear	5

This mapping is important because it is the key to interpreting the classification report and confusion matrix later.

Final Dataset Size After Filtering

After converting to single-label format and filtering to the six chosen emotions, the dataset contained:

- 19,764 training examples
- 4,235 validation examples
- 4,236 test examples

This gives a total of 28,235 samples used in the final project.

7. Train, Validation, and Test Split

The dataset was split into three parts using a 70/15/15 ratio. The first split separated 70% of the data for training and 30% into a temporary set. The temporary set was then split equally into validation and test sets.

Split	Samples
Training	19,764
Validation	4,235
Test	4,236
Total	28,235

This structure is important because each set serves a different role. The training set is used to update the model's parameters. The validation set is used to monitor performance during development and helps detect whether the model is learning appropriately. The test set is used only after training to provide a fair evaluation of final performance.

8. Why BERT Was Chosen

BERT stands for Bidirectional Encoder Representations from Transformers. It is a pre-trained transformer-based model designed to understand language by looking at both the left and right context of each word. Unlike older sequential models such as LSTMs, BERT uses self-attention mechanisms and processes the entire sentence in context.

BERT was chosen because:

1. It is one of the most effective models for text classification tasks.
2. It comes pretrained on large amounts of text, so it already has strong general language knowledge.
3. It can be fine-tuned on a smaller task-specific dataset like GoEmotions.
4. It typically performs much better than traditional machine learning models or older recurrent models on NLP tasks.

This project used `bert-base-uncased`, a pretrained English BERT model provided by Hugging Face.

9. Tokenization

Before training, the text had to be converted into a format BERT could understand. This was done using the BERT tokenizer. Tokenization transformed each sentence into input

IDs, token type IDs, and attention masks — the standard inputs required by BERT.

Padding and truncation were enabled, with a maximum sequence length of 128 tokens. This ensured that all text examples had the same fixed length for training. Shorter sequences were padded, and longer ones were truncated.

10. Creating the Dataset Objects

After tokenization, a custom PyTorch `Dataset` class was created. This class stored the tokenized inputs and labels and returned them in the format expected by the Hugging Face Trainer. Three dataset objects were created: `train_dataset`, `val_dataset`, and `test_dataset`. These objects made it possible to feed the processed data directly into the BERT model during training and evaluation.

11. Fine-Tuning the Pretrained BERT Model

The next stage was model loading and fine-tuning. A `BertForSequenceClassification` model was loaded with `num_labels=6`. This added a classification layer on top of the BERT encoder so that the model could output one of the six emotion classes.

When the model was loaded, the output reported that some weights were unexpected and the classifier weights were newly initialized. This is expected behavior in fine-tuning: the original pretrained BERT checkpoint was built for general pretraining tasks, and the new classification head is randomly initialized before being trained on the task-specific data.

Training Configuration

The model was trained using the Hugging Face `Trainer` API, which simplifies the training loop for transformer models. The hyperparameters used were:

Hyperparameter	Value
Epochs	3
Training batch size	8
Eval batch size	8
Max sequence length	128
Optimizer	AdamW
Learning rate	5e-5 (default)

The number of epochs was set to 3, which provides a good balance between learning and avoiding overfitting for BERT fine-tuning. The batch size of 8 was chosen due to memory constraints when training transformer models in Colab. The AdamW optimizer is widely used for fine-tuning transformer models because it combines adaptive learning rates with effective weight decay. A small learning rate is used to avoid large updates that could

destroy previously learned language representations.

12. Training Process

The training process completed successfully over 3 epochs. The training loss steadily decreased over time, indicating that the model was learning meaningful patterns from the data. Table 1 shows the training loss at selected steps.

Table 1: Training loss at selected steps

Step	Training Loss
500	1.0309
1000	0.8380
1500	0.7998
2000	0.7897
2500	0.7475
3000	0.5893
3500	0.5733
4000	0.6078
4500	0.5443
5000	0.5502
5500	0.3990
6000	0.3957
6500	0.4146
7000	0.3999

```
training_loss=0.6074229700594606, metrics={'train_runtime': 1861.8085, 'train_samples_per_second': 31.846, 'train_steps_per_second': 3.982,
```

The final reported training loss was approximately 0.6074, indicating stable convergence. The total training time was about 1861.81 seconds (approximately 31 minutes), which is reasonable for fine-tuning a BERT model on this dataset size.

13. Evaluation Metrics

The model was evaluated on the test set using the following metrics:

- **Accuracy** — the percentage of predictions that were correct overall.
- **Precision** — how often predicted examples of a class were actually correct.
- **Recall** — how many true examples of a class the model successfully found.
- **F1-score** — the harmonic mean of precision and recall.

- **Per-class F1-score** — especially important in imbalanced datasets, as a model may perform well overall while still performing poorly on individual classes.
- **Confusion matrix** — provides a detailed view of which classes are confused with which other classes.

14. Final Results

The final test-set evaluation returned:

- **Test loss:** 0.9157
- **Accuracy:** 0.7708
- **Weighted F1-score:** 0.7698

```
{'eval_loss': 0.9156861901283264,
 'eval_accuracy': 0.7707743153918791,
 'eval_f1': 0.7697887457084475,
 'eval_runtime': 14.3603,
 'eval_samples_per_second': 294.98,
 'eval_steps_per_second': 36.907,
 'epoch': 3.0}
```

These are strong results for a six-class emotion classification problem. Table 2 shows the full per-class performance.

Table 2: Classification report — BERT-base-uncased

Class	Precision	Recall	F1-score	Support
Sadness (0)	0.74	0.67	0.70	582
Love (1)	0.82	0.86	0.84	820
Joy (2)	0.81	0.72	0.76	816
Surprise (3)	0.71	0.71	0.71	498
Anger (4)	0.78	0.84	0.81	1174
Fear (5)	0.66	0.70	0.68	346
Macro avg	0.75	0.75	0.75	4236
Weighted avg	0.77	0.77	0.77	4236

15. Interpretation of the Results

The overall accuracy of about 77.1% means that the model classified roughly three out of every four sentences correctly. For a six-class emotion classification problem, this is a strong result.

The weighted F1-score being almost equal to the accuracy is a positive sign. It suggests that the model is not performing well only on the largest classes; its performance is relatively balanced when class frequencies are taken into account. The macro F1-score of 0.75 is slightly lower than the weighted F1-score, which indicates that some classes are more difficult than others.

Strongest classes: Love (F1: 0.84) and anger (F1: 0.81) were the best-performing classes. These classes likely benefited from clearer and more distinctive linguistic patterns — text expressing love or anger often contains strong emotional wording.

Moderate classes: Joy (F1: 0.76), surprise (F1: 0.71), and sadness (F1: 0.70) were classified reasonably well but show more overlap with other emotions.

Hardest class: Fear (F1: 0.68) was the weakest class. Fear-related language can overlap with sadness, surprise, or anger, and fear also had the smallest support in the test set (346 examples), meaning the model had fewer training examples to learn from.

16. Confusion Matrix Analysis

The confusion matrix provides a detailed view of the model's behavior. The diagonal contains the correctly classified examples; off-diagonal cells represent misclassifications.

[Insert confusion matrix screenshot here]

The diagonal values were strong overall, consistent with the good accuracy and F1-score. The main confusion patterns observed were:

Sadness: 391 correct out of 582. Most common confusion was with anger (101 instances), followed by fear (38) and joy (22). Some sad sentences contain strong negative language that overlaps with anger.

Love: 706 correct out of 820. Most notable confusion was with joy (60 instances), with minor confusions involving surprise (18) and sadness (13). This is expected since both love and joy are positive emotions with similar language.

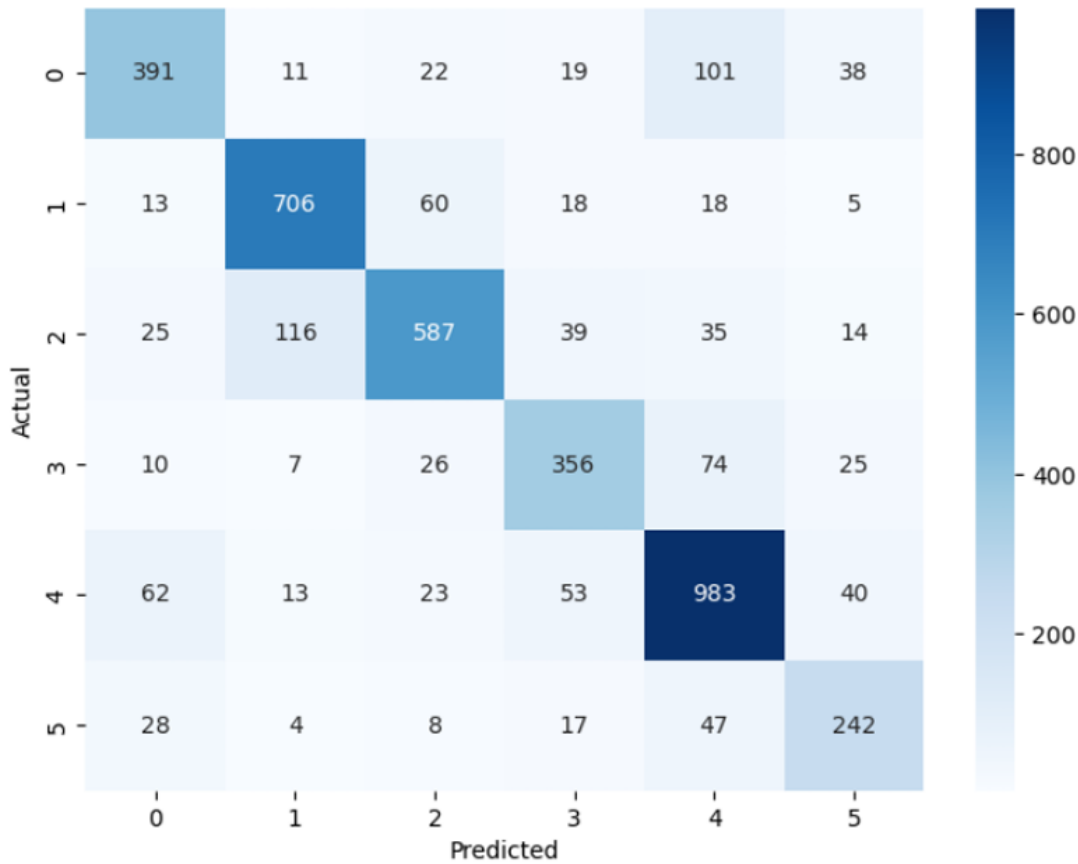
Joy: 587 correct out of 816. Frequently confused with love (116 instances), as well as surprise (39) and anger (35). Joyful and affectionate expressions share significant linguistic overlap.

Surprise: 356 correct out of 498. Strongest confusion was with anger (74 instances),

followed by joy (26) and fear (25). Surprise can be expressed with abrupt or intense language that resembles anger.

Anger: 983 correct out of 1174. Occasionally confused with sadness (62), surprise (53), and fear (40). Negative emotions frequently share similar linguistic features, especially in short text.

Fear: 242 correct out of 346. Most often confused with anger (47 instances), sadness (28), and surprise (17). This confirms fear as the hardest class, likely due to overlap with other intense negative emotions.



17. Error Analysis

The model performs well overall, but several mistakes are understandable because emotion classification is inherently difficult. Human emotions do not always appear in text in a clean or isolated way. The same sentence can carry multiple emotions, and even when only one label is assigned, the wording may still suggest another emotion.

There are several reasons for the observed misclassifications. First, some emotions are semantically similar: joy and love are both positive and often appear in very similar lan-

guage, while sadness, anger, and fear can all involve negative emotional tone. Second, the task was simplified from multi-label to single-label — some sentences originally contained more than one emotion, but the project forced them into one label. Third, class imbalance likely played a role: anger had 1174 test examples while fear had only 346. Fourth, some text samples may be short, ambiguous, sarcastic, or context-dependent, making classification harder for any model.

Overall, the error analysis shows that the model’s mistakes are meaningful rather than random. Most errors happen between classes that genuinely overlap in natural language.

18. Baseline Comparison: BERT vs. DistilBERT

To evaluate whether the full BERT architecture was necessary for this task, a DistilBERT model was trained and evaluated under identical conditions.

What Is DistilBERT?

DistilBERT is a smaller and faster version of BERT, developed by Hugging Face using a technique called knowledge distillation. During distillation, a smaller student model is trained to reproduce the behavior of a larger teacher model — in this case, BERT. The result is a model that retains approximately 97% of BERT’s language understanding performance while using 40% fewer parameters and running roughly 60% faster. DistilBERT has the same general architecture as BERT but with fewer transformer layers, making it a popular choice in settings where speed and memory efficiency matter.

Why DistilBERT Was Chosen for Comparison

DistilBERT was chosen over other candidates such as LSTM or GRU for two reasons. First, it fits directly into the existing pipeline with no modifications — the same tokenizer format, Trainer, and training arguments were reused, meaning the only variable that changed between the two experiments was the model itself. This makes the comparison clean and controlled. Second, DistilBERT is the most meaningful comparison for this project specifically: since the main model is BERT, comparing against DistilBERT directly answers whether the full model capacity was necessary, which is a more focused question than comparing against a fundamentally different architecture like an LSTM.

Experimental Setup

The DistilBERT comparison used the exact same conditions as the main BERT experiment: the same dataset, the same train/validation/test split, the same six emotion classes, the same hyperparameters (3 epochs, batch size 8, maximum sequence length 128, AdamW optimizer), and the same evaluation metrics. The only difference was replacing the BERT model and tokenizer with their DistilBERT equivalents.

Results

	precision	recall	f1-score	support
0	0.73	0.69	0.71	582
1	0.81	0.87	0.84	820
2	0.80	0.69	0.74	816
3	0.73	0.71	0.72	498
4	0.78	0.83	0.81	1174
5	0.66	0.71	0.68	346
accuracy			0.77	4236
macro avg	0.75	0.75	0.75	4236
weighted avg	0.77	0.77	0.77	4236

Table 3 shows the training loss at selected steps.

Table 3: Training loss at selected steps

Step	Training Loss
500	1.029011
1000	0.853748
1500	0.788247
2000	0.779268
2500	0.741644
3000	0.599192
3500	0.578522
4000	0.618686
4500	0.549882
5000	0.561922
5500	0.416694
6000	0.409372
6500	0.430760
7000	0.438694

Table 4 summarizes the results of both models.

Table 4: BERT vs. DistilBERT — test set performance

Model	Accuracy	Weighted F1
BERT-base-uncased	0.7708	0.7698
DistilBERT-base-uncased	0.7682	0.7669
Difference	+0.0026	+0.0029

Interpretation

DistilBERT achieved an accuracy of 76.82% and a weighted F1-score of 76.69%, compared to BERT’s 77.08% accuracy and 76.98% weighted F1. The difference is approximately 0.26 percentage points in accuracy and 0.29 points in weighted F1, which is a very small margin. The per-class performance of DistilBERT was also nearly identical to BERT across all six emotion classes — both models showed the same difficulty ranking, with love and anger as the strongest classes and fear as the hardest.

This comparison confirms two things. First, the full BERT model does provide a slight but consistent advantage over DistilBERT, justifying the architectural choice made in this project. Second, DistilBERT is a remarkably competitive alternative that achieves almost the same performance while being significantly faster and lighter. In a production environment where inference speed or hardware constraints matter, DistilBERT would be a very reasonable choice with minimal performance cost.

19. What Was Successfully Achieved

This project successfully completed all of the planned objectives. The implementation:

- Used the GoEmotions dataset as the source of labeled data.
- Converted the data from multi-label to single-label format using a clear and reproducible rule.
- Focused on six specific emotion classes.
- Loaded and fine-tuned a pretrained BERT model.
- Evaluated the model with accuracy, F1-score, and per-class metrics.
- Included a confusion matrix and detailed error analysis.
- Compared BERT against a DistilBERT baseline under controlled conditions.

This means the project fully matched the approved proposal and the instructor’s requested scope.

20. Limitations

Although the project was successful, it has several limitations.

The biggest limitation is the conversion from multi-label to single-label classification. This simplification made training easier, but it also removed emotional complexity from the original dataset. Some sentences naturally express more than one emotion, and forcing them into a single label introduces noise into both training and evaluation.

A related issue is that the choice of primary emotion was based on the first active label in the dataset’s column order. This is a practical rule, but it may not always represent the strongest or most important emotion in the sentence, and it introduces a systematic bias tied to column ordering rather than emotional salience.

A third limitation is that only six emotions were used instead of the full GoEmotions label set. This made the task more manageable, but it reduced the richness and real-world applicability of the final model.

A further limitation is the absence of a neutral class. The original GoEmotions dataset includes a neutral label for sentences that do not express any clear emotion, but this project excluded it in order to focus on the six target emotions. As a result, the model is always forced to assign one of the six emotion classes to every input, even when the sentence carries no emotional content at all. For example, a factual statement such as “*The meeting is at 3 PM*” will receive a prediction, but that prediction is meaningless because no emotion is actually present.

Finally, class imbalance was not explicitly addressed. Anger had 1174 test examples while fear had only 346, which likely contributed to the weaker performance on fear.

21. Possible Improvements

If this project were extended in the future, several improvements could be explored.

A more advanced version could keep the task as true multi-label classification rather than converting it to single-label, which would better reflect the original nature of GoEmotions and avoid the information loss introduced by the simplification.

Another improvement would be to apply class balancing techniques or weighted loss functions to help underrepresented classes such as fear perform better.

Including a neutral class would also significantly improve the practical applicability of the system, allowing it to correctly handle sentences with no emotional content rather than forcing a spurious prediction.

22. Conclusion

This project demonstrated a complete deep learning workflow for NLP using a fine-tuned BERT model for emotion detection. Starting from the GoEmotions dataset, the data was converted from multi-label format into a single-label classification problem by selecting the first active emotion per sentence. The dataset was then filtered to six chosen emotions, tokenized with the BERT tokenizer, and used to fine-tune a pretrained `bert-base-uncased` model.

The final model achieved approximately 77.1% accuracy and 77.0% weighted F1-score on the test set, which indicates strong performance for this six-class emotion detection task. A DistilBERT baseline trained under identical conditions achieved 76.82% accuracy and 76.69% weighted F1, confirming that full BERT provides a consistent advantage while DistilBERT remains a competitive and efficient alternative.

The per-class analysis showed that love and anger were the easiest emotions to classify, while fear was the most difficult. The confusion matrix and error analysis showed that most mistakes occurred between semantically similar emotions, which is expected in fine-grained emotion classification.

Overall, the project was successful both technically and conceptually. It demonstrated understanding of transformers, pretrained language models, fine-tuning, dataset preprocessing, evaluation metrics, and result interpretation. It also showed how practical implementation choices — such as converting multi-label data into single-label format — affect both the training process and the final performance of an NLP system.

Appendix: Summary of the Full Implementation Flow

The implementation followed these steps in order. The CSV dataset was uploaded into Google Colab and loaded with pandas. The dataset structure was inspected to confirm the presence of the text column and the one-hot encoded emotion columns. The emotion columns were selected, and a new label column was created using `idxmax(axis=1)` so that the first active emotion became the primary label. The dataset was filtered to keep only six emotion classes: sadness, love, joy, surprise, anger, and fear. Only the text and label columns were retained. The label names were mapped to numeric IDs. The cleaned dataset was split into training, validation, and test sets. The text data was tokenized using the BERT tokenizer with truncation, padding, and a maximum sequence length of 128. Custom dataset objects were built using PyTorch. A pretrained model with a six-class classification head was loaded. Training arguments were configured, and a metrics function was defined to compute accuracy and weighted F1-score. The model was then fine-tuned for 3 epochs using the Hugging Face Trainer. After training, the model was evaluated on the test set, and detailed performance was examined using a classification report and confusion matrix. Finally, a DistilBERT model was trained and evaluated under identical conditions as a controlled baseline comparison.

GitHub Repository: https://github.com/AssilHalawi/Emotion_detection